Team Involvement in Demonstration

Date	29/06/2026
Team ID	Not Assigned
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	1 Mark

Team Involvement in Demonstration:

This document records the active participation and roles of each team member during the project demonstration. It ensures that every member contributes meaningfully to the presentation and that responsibilities are distributed fairly and clearly.

S. No	Team Member Name	Role in Demo	Section Presented	Contribution Summary	Participation (Active / Passive)
1	Gowthamsai Setti	Team Lead & Main Presenter	Section 1: Introduction & Problem Statement; Section 3: Data Preprocessing & Model Building (co-presenter); Section 5: Flask app.py routes & integration walkthrough	Led overall project coordination across all 4 sprints. Built app.py with all Flask routes, integrated XGBoost model and StandardScaler at startup. Opened the demo with project introduction, co-presented preprocessing pipeline, and walked through Flask integration in the code section.	Active
2	Rishitha Padaga	Frontend Developer & Demo Operator	Section 2: EDA Walkthrough (univariate analysis co-presenter); Section 4: Live Web App Demonstration — operated Render.com live app, entered sample inputs, showed flood/no-flood result pages	Designed and developed all 4 Jinja2 HTML templates (home.html, index.html, chance.html, no_chance.html) with dark-themed CSS. Performed univariate analysis in Sprint-1. Operated the live demo on Render.com during presentation.	Active
3	Sai Siddardha Bevara	ML Engineer & Model Presenter	Section 3: Data Preprocessing & Model Building — presented full preprocessing pipeline, trained all 4 classifiers (Decision Tree, Random Forest, KNN, XGBoost), showed model comparison and explained XGBoost selection (96.55% accuracy)	Led Sprint-2 (data preprocessing) and Sprint-3 (model building). Trained Decision Tree, Random Forest, and XGBoost classifiers. Presented confusion matrix, F1-score, ROC-AUC comparison. Explained scale_pos_weight for class imbalance handling.	Active
4	Karri Dhinesh	Data Analyst & EDA Presenter	Section 2: Dataset & EDA Walkthrough — presented flood dataset.xlsx structure, all 10 features, distribution plots, box plots, correlation heatmap, violin plots and descriptive statistics from flood_eda.ipynb	Performed full EDA in flood_eda.ipynb. Imported all Python libraries and explored dataset structure (head(), shape, info(), describe()). Presented EDA visualizations — distribution plots, box plots, and correlation heatmap — during the demo.	Active

S. No	Team Member Name	Role in Demo	Section Presented	Contribution Summary	Participation (Active / Passive)
5	Ravi Kumar Yadla	DevOps Engineer & Deployment Lead	Section 5: Code, GitHub & Deployment Walkthrough — showed project file structure, GitHub repo commit history, Procfile, Render.com auto-deploy pipeline; Section 6: Performance Testing — presented k6 load test results	Implemented KNN classifier and model comparison in Sprint-3. Created Procfile, pushed project to GitHub, and deployed Flask app on Render.com in Sprint-4. Presented deployment architecture, GitHub CI/CD pipeline, and k6 performance test results (20 VUs, 95th percentile < 2s).	Active
6	All Members	Q&A; Responders	Section 6: Q&A; Session — each member answered evaluator questions in their sprint domain area	All 5 members collectively responded to evaluator questions. Domain split: EDA/data (Karri Dhinesh, Rishitha Padaga), preprocessing/model building (Sai Siddardha Bevara, Gowthamsai Setti), deployment/performance (Ravi Kumar Yadla), Flask integration/overall architecture (Gowthamsai Setti).	Active

Team Coordination Notes:

Aspect	Details
Team Leader / Coordinator	Gowthamsai Setti — Led sprint planning across all 4 sprints, managed GitHub repository (owned app.py and Flask route integration), coordinated daily WhatsApp standups, and drove end-to-end integration of XGBoost model, StandardScaler, and Flask application.
Overall Team Coordination Rating (1-5)	5 / 5 — All 5 members completed their sprint tasks on time across the 4-day development cycle (24–27 June 2026). Daily WhatsApp standups, shared Google Drive for notebook outputs, and clear file ownership (Gowthamsai: app.py; Rishitha: HTML templates) prevented conflicts.
Any issues during demo	Minor cold-start delay (~30–60 seconds) observed when loading the Render.com free-tier instance at the start of the demo session. No functional failures — all 4 routes, prediction logic, probability scores, and result pages worked correctly throughout.
How issues were resolved	Team pre-loaded the Render.com application before the demo session began to eliminate cold-start delay. A backup local Flask instance (http://127.0.0.1:5000) was kept running as a fallback. No fallback was needed — the live app performed within the 2-second threshold during the demo.